JC2001: Introduction to Software Engineering

Lecture 8: Software Architecture, Design and Implementation

Lecture Schedule: Week 7, Day 1

Dr Zhenhua Yu zhenhua.yu@abdn.ac.uk

Dr Riazul Islam riazul.islam@abdn.ac.uk

Dr Shahzad Mumtaz shahzad.mumtaz@abdn.ac.ukc

Outline

Architectural design (Chapter 6)

- Architectural design decisions
- Architectural views
- Architectural patterns
- Application architectures

Design and implementation (Chapter 7)

- Object-oriented design using the UML
- Design patterns
- Implementation issues
- Open-source development

Architectural design

1495 UNIVERSITY OF ABERDEEN

Architectural design

- Architectural design is concerned with understanding how a software system should be organized and designing the overall structure of that system.
- In agile processes, it is generally accepted that an early stage of an agile development process should focus on designing an overall system architecture.

Architectural design

Software architectures can be designed at two levels of abstraction, which I call architecture in the small and architecture in the large:

- **Architecture in the small** is concerned with the architecture of individual programs. At this level, we are concerned with the way that an individual program is decomposed into components. This chapter is mostly concerned with program architectures.
- **Architecture in the large** is concerned with the architecture of complex enterprise systems that include other systems, programs, and program components. These enterprise systems may be distributed over different computers, which may be owned and managed by different companies. (Chapters 17 and 18.)
- NB: Non-functional requirements had the most significant effect on the system's architecture.

Architectural design

Software architectures can be designed at two levels of abstraction, which I call architecture in the small and architecture in the large:

- **Architecture in the small** is concerned with the architecture of individual programs. At this level, we are concerned with the way that an individual program is decomposed into components. This chapter is mostly concerned with program architectures.
- **Architecture in the large** is concerned with the architecture of complex enterprise systems that include other systems, programs, and program components. These enterprise systems may be distributed over different computers, which may be owned and managed by different companies. (Chapters 17 and 18.)
- NB: Non-functional requirements had the most significant effect on the system's architecture.

Architectural design

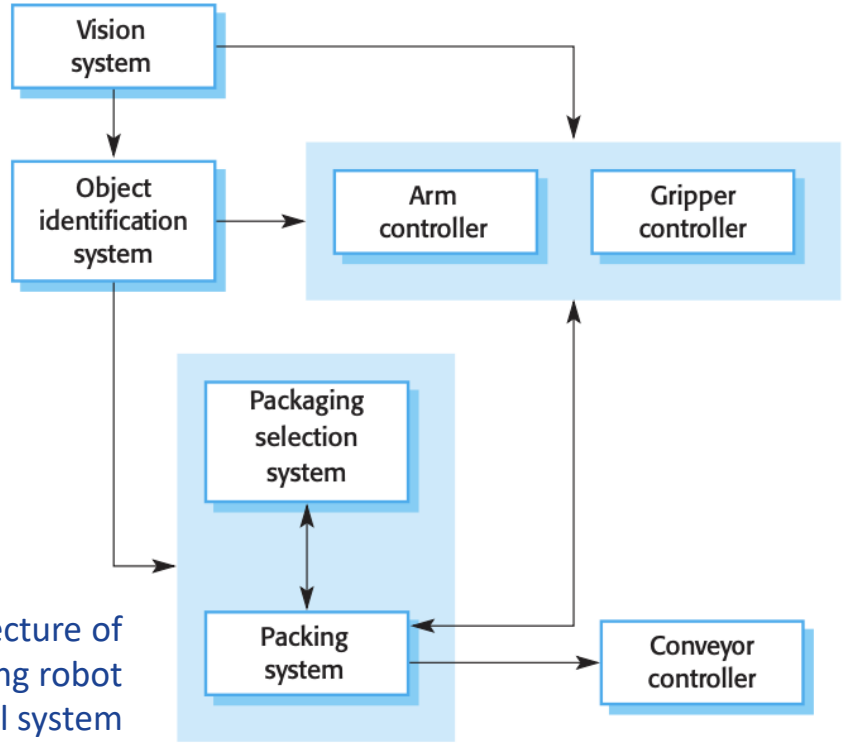
Advantages of explicitly designing and documenting software architecture:

- **Stakeholder communication** -- The architecture is a high-level presentation of the system that may be used as a focus for discussion by a range of different stakeholders.
- **System analysis** -- Making the system architecture explicit at an early stage in the system development requires some analysis. Architectural design decisions have a profound effect on whether or not the system can meet critical requirements such as performance, reliability, and maintainability.
- **Large-scale reuse** -- The system architecture is often the same for systems with similar requirements and so can support large-scale software reuse. *Product-line architectures* are an approach to reuse where the same architecture is reused across a range of related systems (Chapter 15).

Architectural design

System architectures are often modeled informally using simple block diagrams.

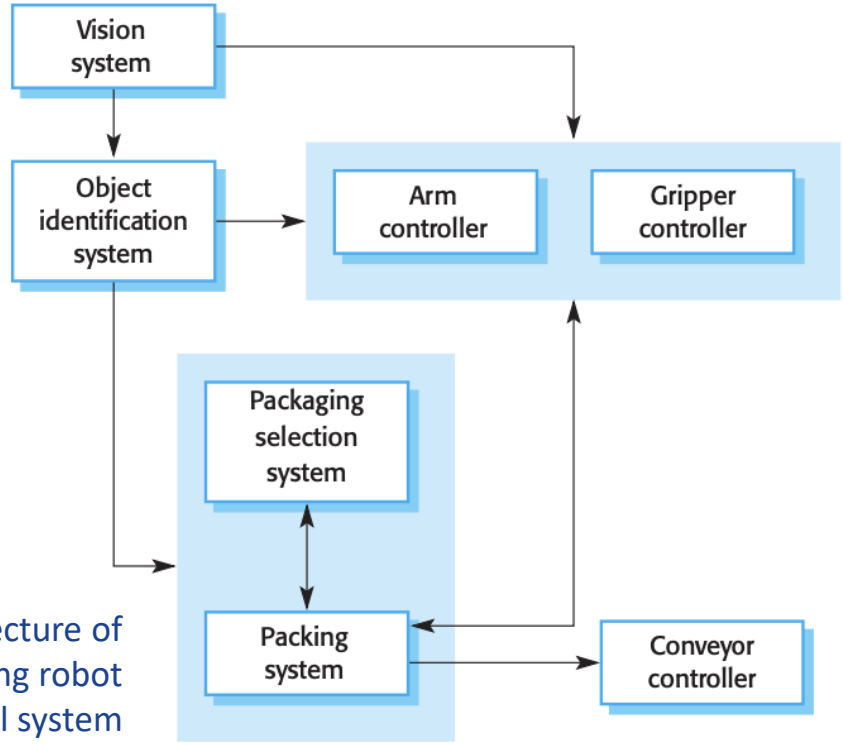
- Each box in the diagram represents a component.
- Boxes within boxes indicate that the component has been decomposed to subcomponents.
- Arrows mean that data and or control signals are passed from component to component in the direction of the arrows.



The architecture of a packing robot control system

Architectural design

- Very abstract - they do not show the nature of component relationships nor the externally visible properties of the sub-systems.
- However, useful for communication with stakeholders and for project planning.



The architecture of a packing robot control system

Architectural design

Use of architectural models

- As a way of facilitating discussion about the system design
  A high-level architectural view of a system is useful for communication with system stakeholders and project planning because it is not cluttered with detail. Stakeholders can relate to it and understand an abstract view of the system. They can then discuss the system as a whole without being confused by detail.
- As a way of documenting an architecture that has been designed
  The aim here is to produce a complete system model that shows the different components in a system, their interfaces and their connections.

Mock quiz

- Which one of the following is an advantage of designing and documenting a software architecture?

A. A software architecture is a high-level presentation of the system which can be used as a focus for discussion by a range of different stakeholders.

B. A software architecture can be used to identify errors or bugs in the program.

C. A software architecture can be used to provide small-scale reuse.

D. A software architecture supports decision making.

Architectural design decisions

Architectural design is a **creative process** so the process **differs depending on the type of system** being developed.

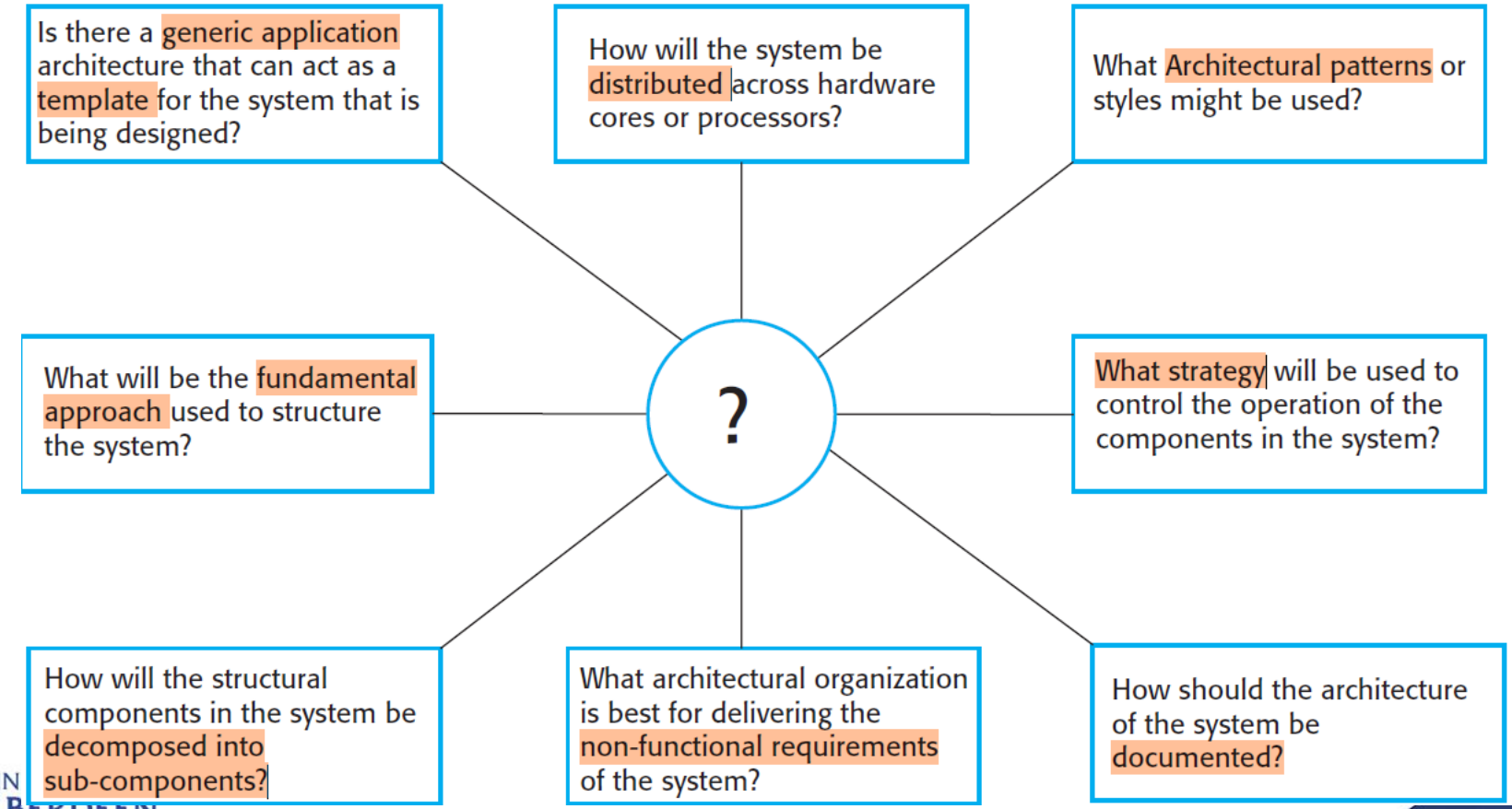
However, a number of **common decisions span all design processes** and these decisions affect the non-functional characteristics of the system.

Architectural design decisions

During the architectural design process, system architects have to make a number of structural decisions that profoundly affect the system and its development pro- cess.

- Is there a generic application architecture that can act as a template for the system that is being designed?
- How will the system be distributed across hardware cores or processors?
- What Architectural patterns or styles might be used?
- What will be the fundamental approach used to structure the system?
- What strategy will be used to control the operation of the components in the system?
- How will the structural components in the system be decomposed into sub-components?
- **What architectural organization is best for delivering the non-functional requirements of the system?**
- How should the architecture of the system be documented?

Architectural design decisions



Architectural design decisions

- …
- **What architectural organization is best for delivering the non-functional requirements of the system?**
    - **Performance** -- Localise critical operations and minimise communications. Use large rather than fine-grain components.
    - **Security** -- Use a layered architecture with critical assets in the inner layers.
    - **Safety** -- Localise safety-critical features in a small number of sub-systems.
    - **Availability** -- Include redundant components and mechanisms for fault tolerance.
    - **Maintainability** -- Use fine-grain, replaceable components.
- …

Architectural design decisions

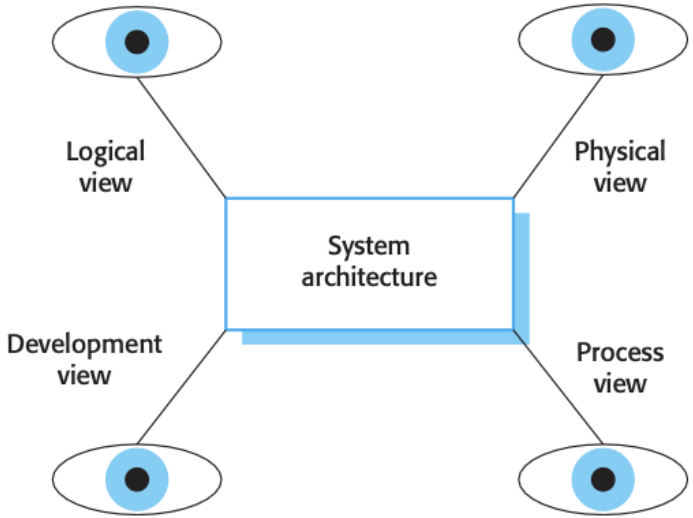
- There is potential conflict between some of these architectures. For example, using large components improves performance, and using small, fine-grain components improves maintainability. If both performance and maintainability are important system requirements, however, then some compromise must be found. You can sometimes do this by using different Architectural patterns or styles for separate parts of the system.
- Evaluating an architectural design is difficult because the true test of an architecture is how well the system meets its functional and non-functional requirements when it is in use. However, you can do some evaluation by comparing your design against reference architectures or generic Architectural patterns.

Architectural views

Each architectural model only shows one view or perspective of the system.

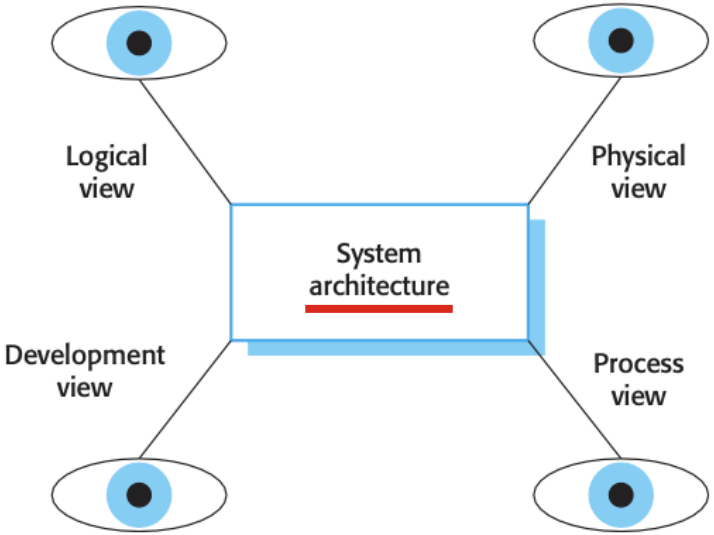
- It might show how a system is decomposed into modules, how the run-time processes interact or the different ways in which system components are distributed across a network.
- For both design and documentation, you usually need to present multiple views of the software architecture.

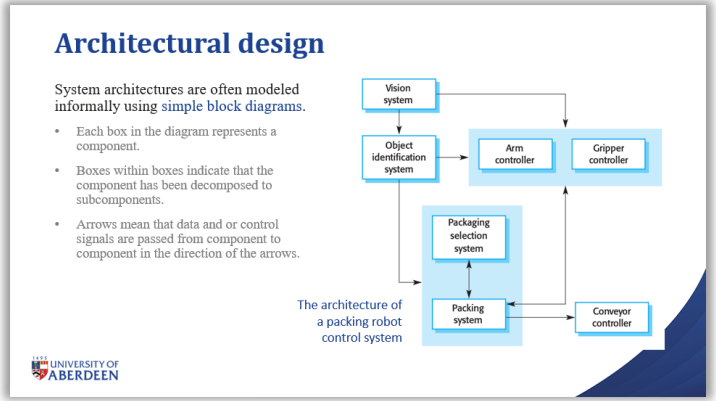
4+1 views or perspectives are useful when designing and documenting a system's architecture.



Architectural views

4+1 views or perspectives are useful when designing and documenting a system's architecture.

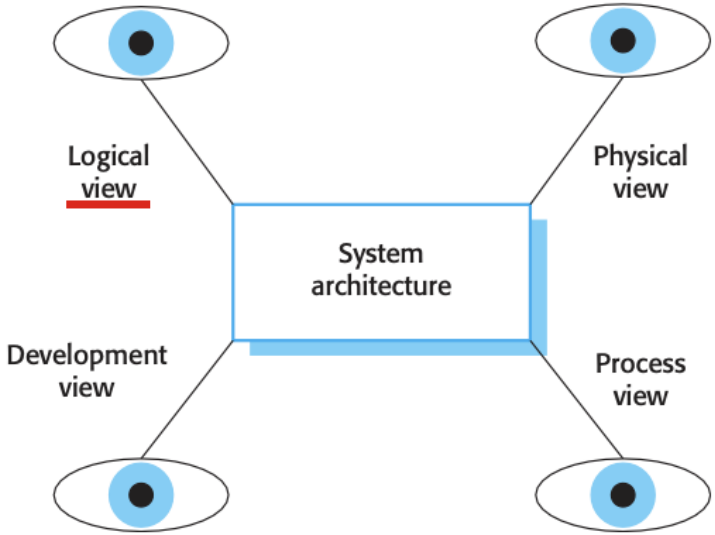




(System architecture example)

Architectural views

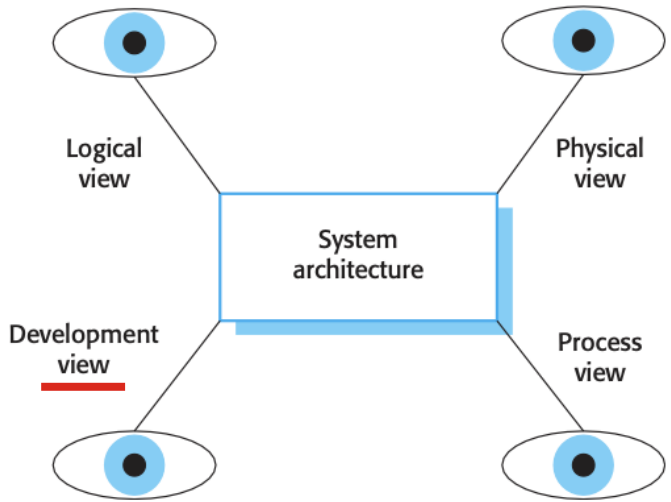
4+1 views or perspectives are useful when designing and documenting a system's architecture.



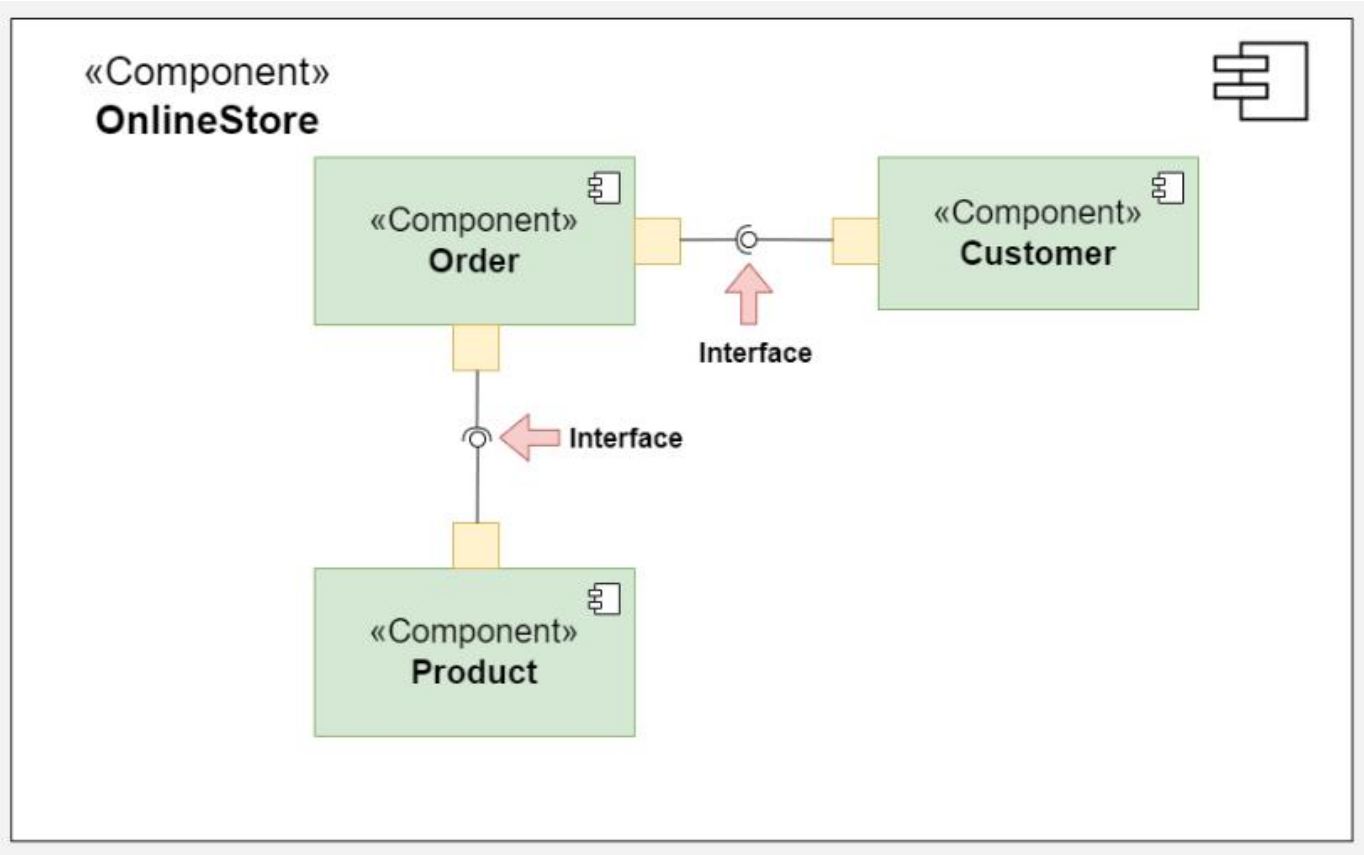
- A **logical view**, which shows the key abstractions in the system as objects or object classes.
- It should be possible to relate the system requirements to entities in this logical view.

Architectural views

4+1 views or perspectives are useful when designing and documenting a system's architecture.

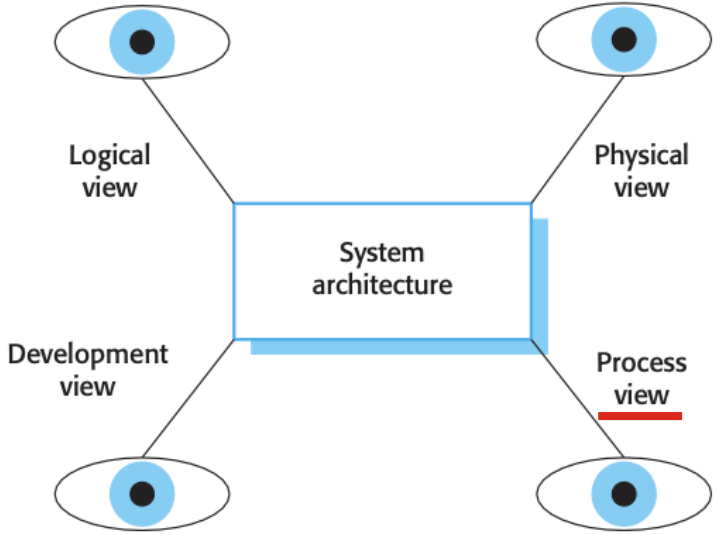


- A **development view**, which shows how the software is decomposed for development; that is, it shows the breakdown of the software into components that are implemented by a single developer or development team.
- This view is useful for software managers and programmers.


«Component»
OnlineStore
«Component»
Order
«Component»
Customer
Interface
Interface
«Component»
Product

Architectural views

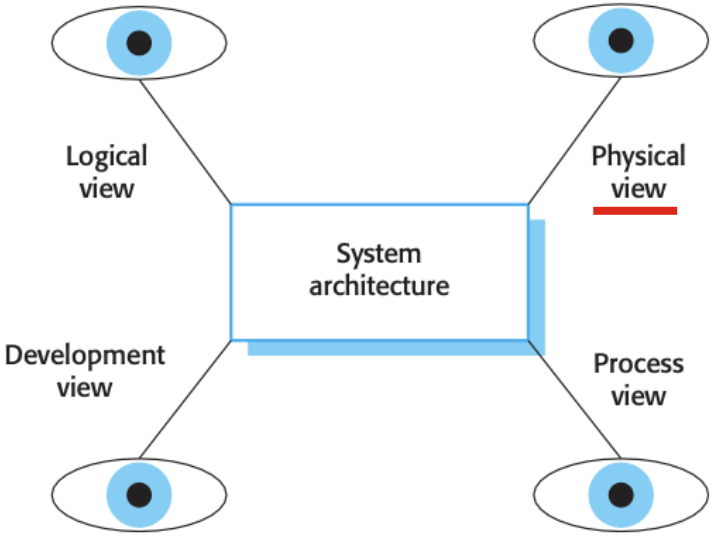
4+1 views or perspectives are useful when designing and documenting a system's architecture.



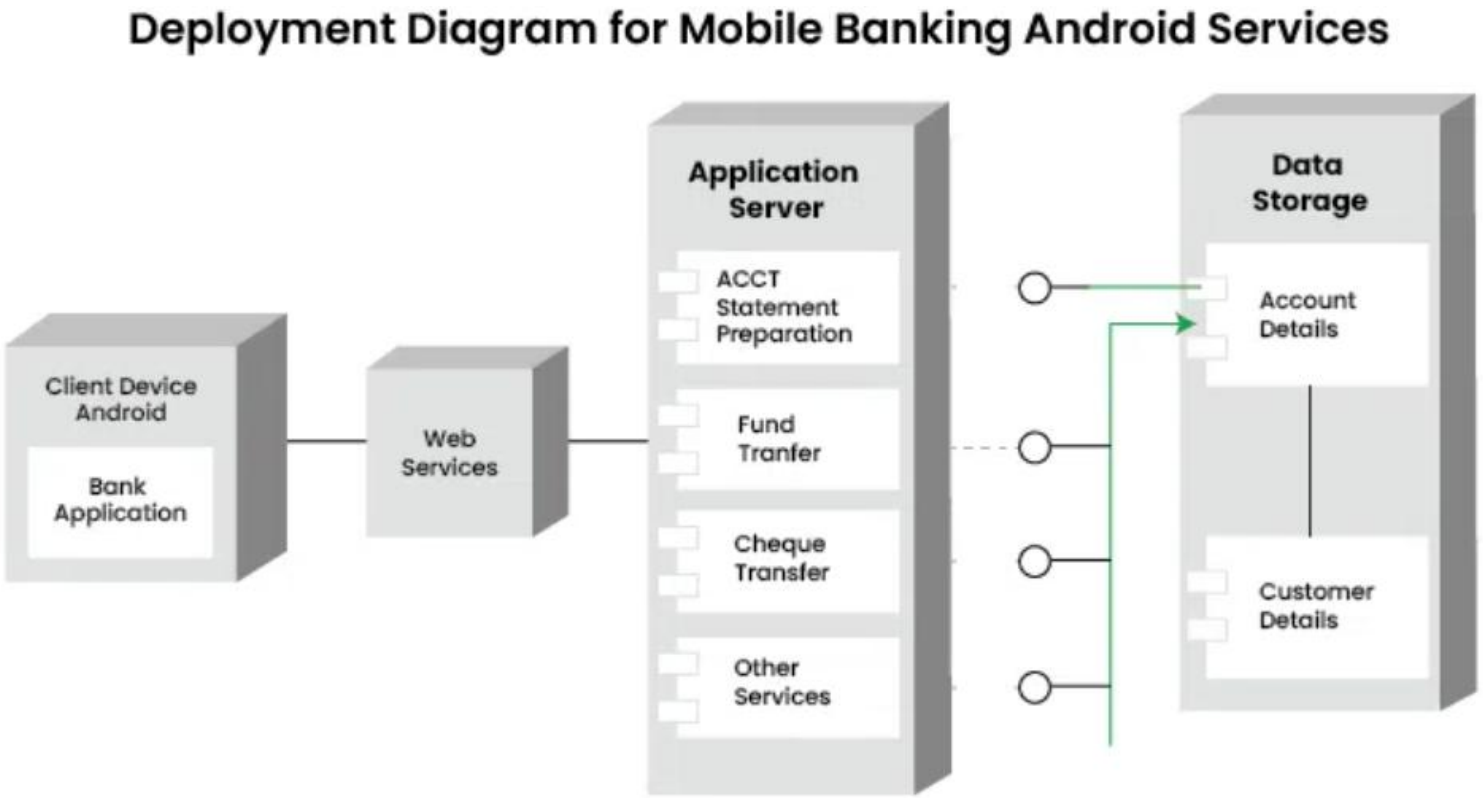
- A **process view**, which shows how, at runtime, the system is composed of interacting processes.
- This view is useful for making judgments about non-functional system characteristics such as performance and availability.

Architectural views

4+1 views or perspectives are useful when designing and documenting a system's architecture.



- A **physical view**, which shows the system hardware and how software components are distributed across the processors in the system.
- This view is useful for systems engineers planning a system deployment.



https://www.geeksforgeeks.org/system-design/deployment-diagram-unified-modeling-languageuml/

Architectural views

4+1 views or perspectives are useful when designing and documenting a system's architecture.

Some of the other views may also be developed when different aspects of the system are discussed, although it is rarely necessary to develop a complete description from all perspectives.

Architectural views

Notations used for describing architectural models:

- Informal models and notations – (our textbook recommendation)
    - as long as it's clear
- Unified Modeling Language (UML)
    - The UML was designed for describing object-oriented systems (rather than architecture).
    - The UML is of most value when you are documenting an architecture in detail or using model-driven development.
- Architectural Description Languages (ADLs)
    - Because ADLs are specialist languages, domain and application specialists find it hard to understand and use ADLs.
- …

Mock quiz

- Give one reason why UML is unsuitable for describing architectural models?

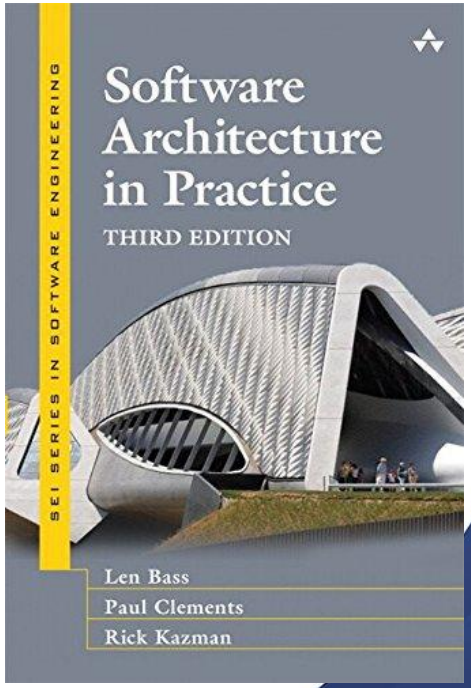
A. UML was designed for describing object-oriented systems.
B. UML was designed to be a standard for describing a system's architecture.
C. UML was designed for specifying a system's components.
D. UML was designed for describing modern computer architectures.

Architectural patterns

An architectural pattern describes a system organization that has been successful in previous systems.

It should include information on when it is and is not appropriate to use that pattern, and details on the pattern's strengths and weaknesses.

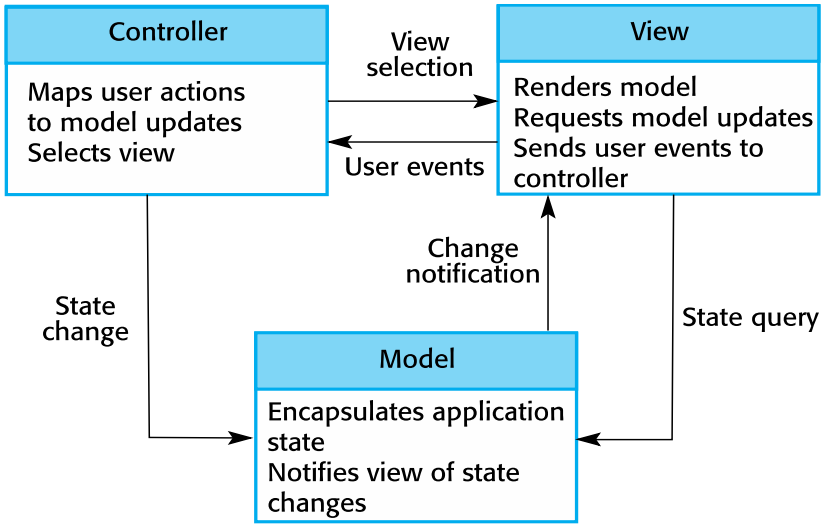
For more detailed information about patterns and their use, you should refer to a published pattern handbooks. We will only introduce some typical architectural patterns.



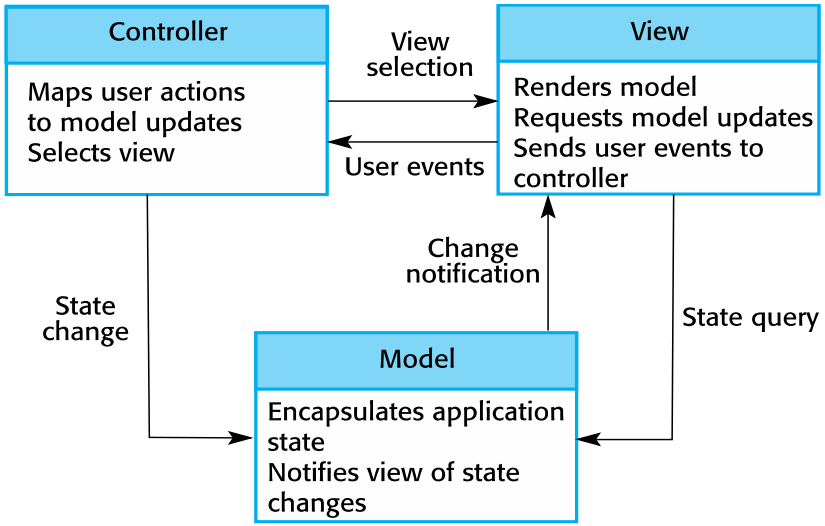
Architectural patterns
-- Model-View-Controller (MVC) pattern

Description

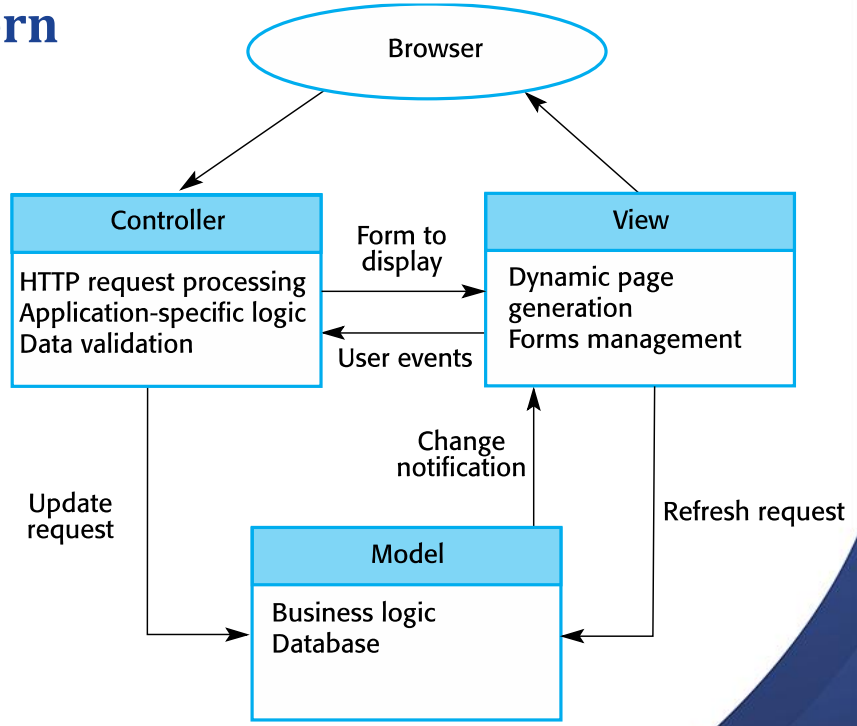
- Separates presentation and interaction from the system data. The system is structured into three logical components that interact with each other. The **Model component** manages the system data and associated operations on that data. The **View component** defines and manages how the data is presented to the user. The **Controller component** manages user interaction (e.g., key presses, mouse clicks, etc.) and passes these interactions to the View and the Model.



Architectural patterns
-- Model-View-Controller (MVC) pattern



The organization of the Model-View-Controller



Web application architecture using the MVC pattern

Architectural patterns
-- Model-View-Controller (MVC) pattern

When used

- When there are multiple ways to view and interact with data. Also used when the future requirements for interaction and presentation of data are unknown.

Advantages

- Allows the data to change independently of its representation and vice versa. Supports presentation of the same data in different ways with changes made in one representation shown in all of them.

Disadvantages

- Can involve additional code and code complexity when the data model and interactions are simple.

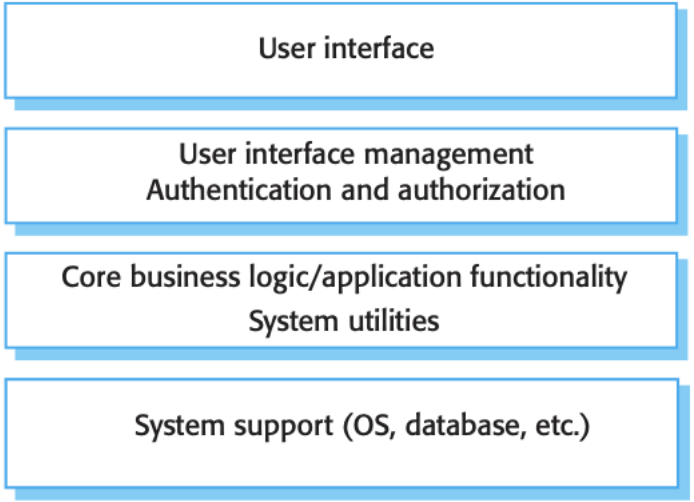
Architectural patterns
-- Layered architecture

Description

- Organizes the system into layers, with related functionality associated with each layer.
- A layer provides services to the layer above it, so the lowest level layers represent core services that are likely to be used throughout the system.

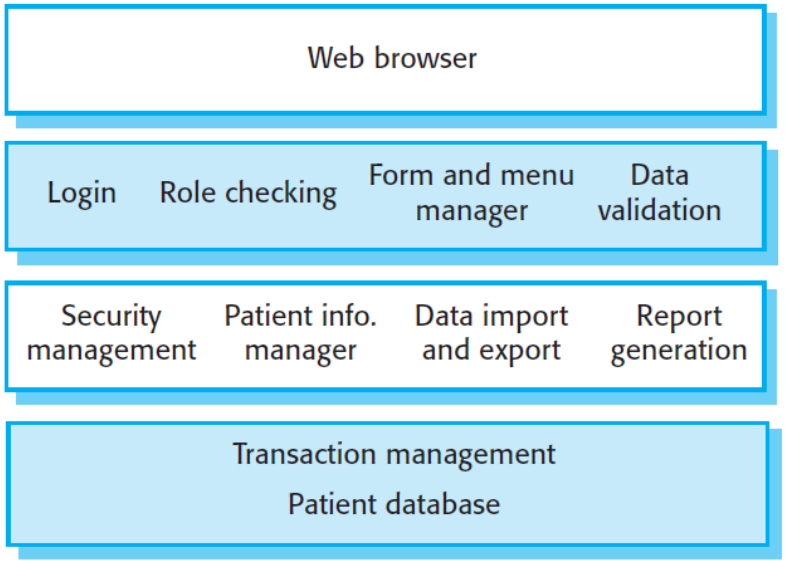
When used

- when building new facilities on top of existing systems;
- when the development is spread across several teams with each team responsibility for a layer of functionality;
- when there is a requirement for multilevel security.



Layered architecture: Example Mentcare system

- The top layer is a browser-based user interface, and the second layer provides the user interface functionality that is delivered through the web browser
- The third layer implements the functionality of the system and provides components that implement…
- The lowest layer, which is built using a commercial database management system, provides transaction management and persistent data storage.



Architectural patterns
-- Layered architecture

Advantages

- Allows replacement of entire layers as long as the interface is maintained.
- Redundant facilities (e.g., authentication) can be provided in each layer to increase the dependability of the system.

Disadvantages

- In practice, providing a clean separation between layers is often difficult, and a high-level layer may have to interact directly with lower-level layers rather than through the layer immediately below it.
- Performance can be a problem because of multiple levels of interpretation of a service request as it is processed at each layer.

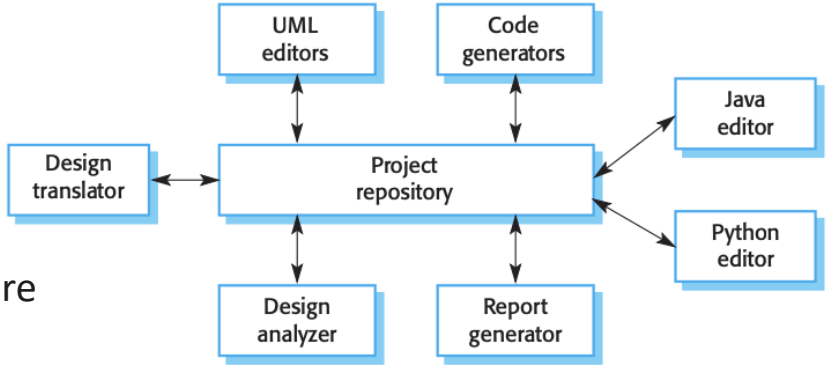
Architectural patterns
-- Repository architecture

Description

- All data in a system is managed in a central repository that is accessible to all system components.
- Components do not interact directly, only through the repository.

When used

- When a system in which large volumes of information are generated that has to be stored for a long time.
- You may also use it in data-driven systems where the inclusion of data in the repository triggers an action or tool.



Architectural patterns
-- Repository architecture

Advantages

- Components can be independent; they do not need to know of the existence of other components. Changes made by one component can be propagated to all components.
- All data can be managed consistently (e.g., backups done at the same time) as it is all in one place.

Disadvantages

- The repository is a single point of failure so problems in the repository affect the whole system.
- May be inefficiencies in organizing all communication through the repository. Distributing the repository across several computers may be difficult.

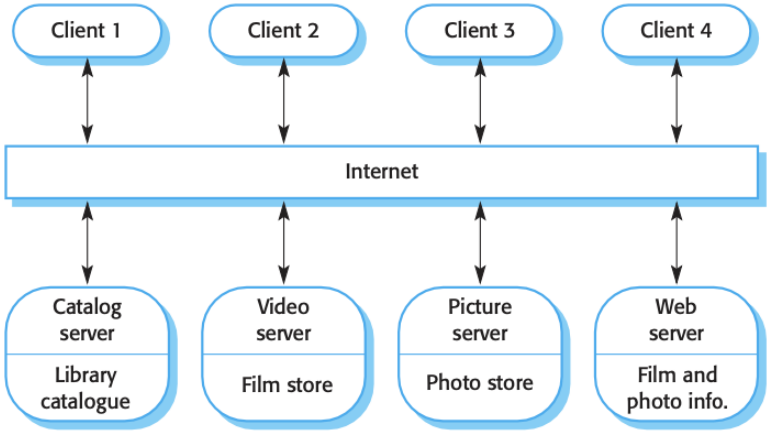
Architectural patterns
-- Client–server architecture

Description

- The system is presented as a set of services, with each service delivered by a separate server. Clients are users of these services and access servers to make use of them.

When used

- When data in a shared database has to be accessed from a range of locations. Because servers can be replicated, may also be used when the load on a system is variable.



Architectural patterns
-- Client–server architecture

Advantages

- Servers can be distributed across a network.
- General functionality (e.g., a printing service) can be available to all clients and does not need to be implemented by all services.

Disadvantages

- Each service is a single point of failure and so is susceptible to denial-of-service attacks or server failure.
- Performance may be unpredictable because it depends on the network as well as the system.
- Management problems may arise if servers are owned by different organizations.

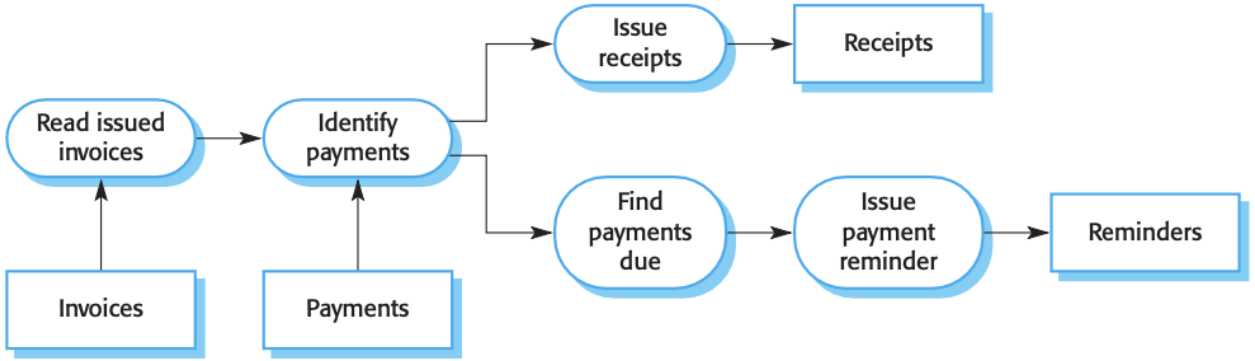
Architectural patterns
-- Pipe and filter architecture

Description

- The processing of the data in a system is organized so that each processing component (filter) is discrete and carries out one type of data transformation. The data flows (as in a pipe) from one component to another for processing.

When used

- Commonly used in data-processing applications (both batch and transaction-based) where inputs are processed in separate stages to generate related outputs.



Architectural patterns
-- Pipe and filter architecture

Advantages

- Easy to understand and supports transformation reuse.
- Workflow style matches the structure of many business processes.
- Evolution by adding transformations is straightforward.
- Can be implemented as either a sequential or concurrent system.

Disadvantages

- The format for data transfer has to be agreed between communicating transformations. Each transformation must parse its input and unparse its output to the agreed form. This increases system overhead and may mean that it is impossible to reuse architectural components that use incompatible data structures.

Mock quiz

- Which of the following are disadvantages of the client-server architecture?

A. Each service is a single point of failure and is susceptible to denial-of-service attacks.

B. Performance may be unpredictable because it depends on the network as well as the system.

C. Management problems may arise if servers are owned by different organisations.

D. All of the above.

Design and implementation

1495 UNIVERSITY OF ABERDEEN

Object-oriented design using the UML

To develop a system design from concept to detailed (object-oriented design), you need to:

1. Understand and define the context and the external interactions with the system.
2. Design the system architecture.
3. Identify the principal objects in the system.
4. Develop design models.
5. Specify interfaces.

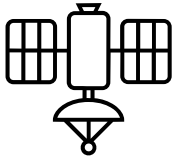
We will use a concrete example, an **Weather Station System**, to explain the design process.

The wilderness weather system

- A national weather service wishes to collect weather information from remote areas to help with weather forecasting.
- The intention therefore is to develop remote automatic collection systems that are connected to a broader weather information system.

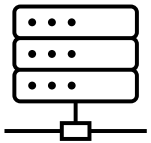
Weather Information System

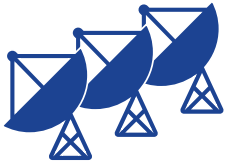




Satellite

Control System





Weather Stations

The wilderness weather system
-- A use case example (already collected)

System **Weather station**

Use case Report weather

Actors Weather information system, Weather station

Data The weather station sends a summary of the weather data that has been collected from the instruments in the collection period to the weather information system. E.g. air temperatures, air pressures, wind speeds etc.

Stimulus The *weather information system* establishes a satellite communication link with the weather station and requests transmission of the data.

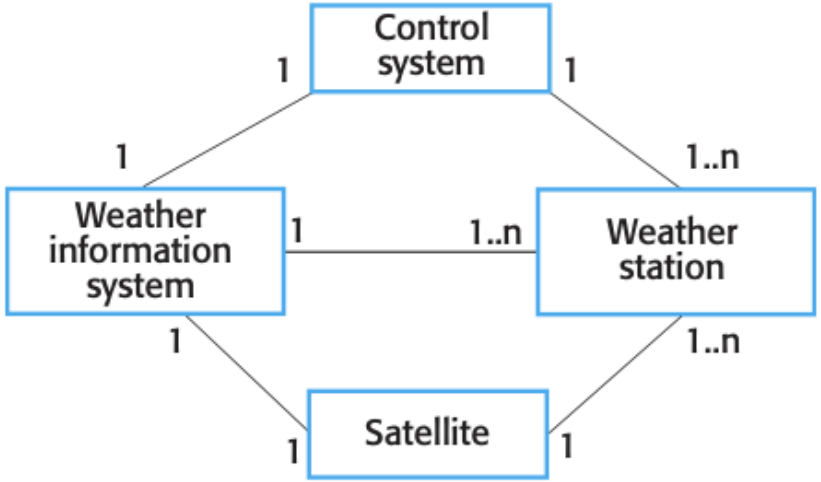
Response The data is sent to the weather information system.

Comments Weather stations are usually asked to report once per hour, but this frequency may differ from one station to another and may be modified in future.

Object-oriented design using the UML

1. System context and interactions

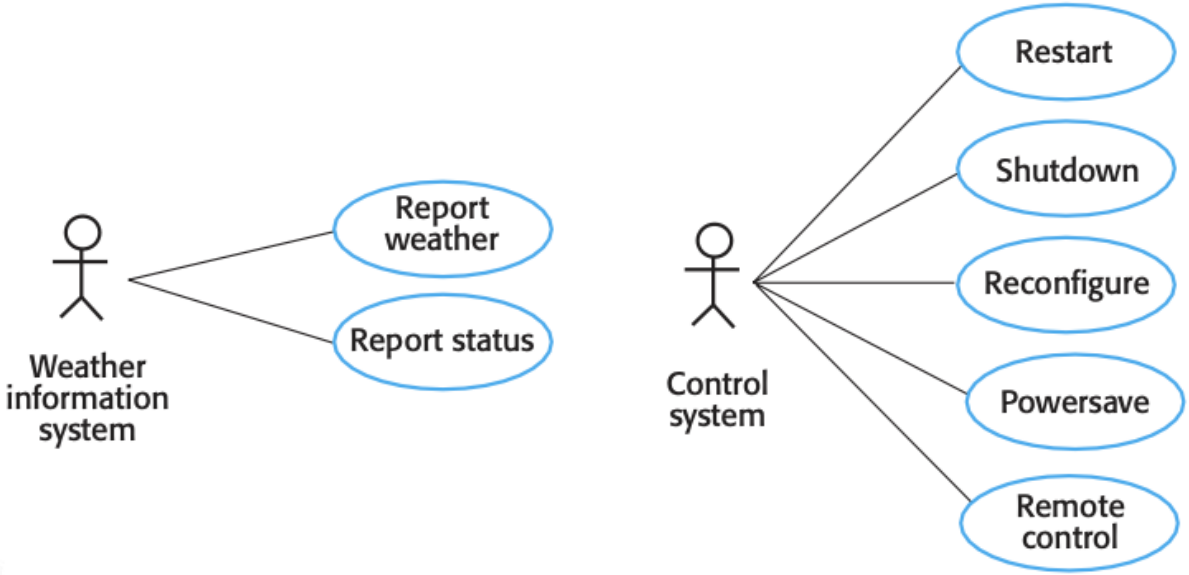
A system context model is a structural model that demonstrates the other systems in the environment of the system being developed.



Object-oriented design using the UML

1. System context and interactions

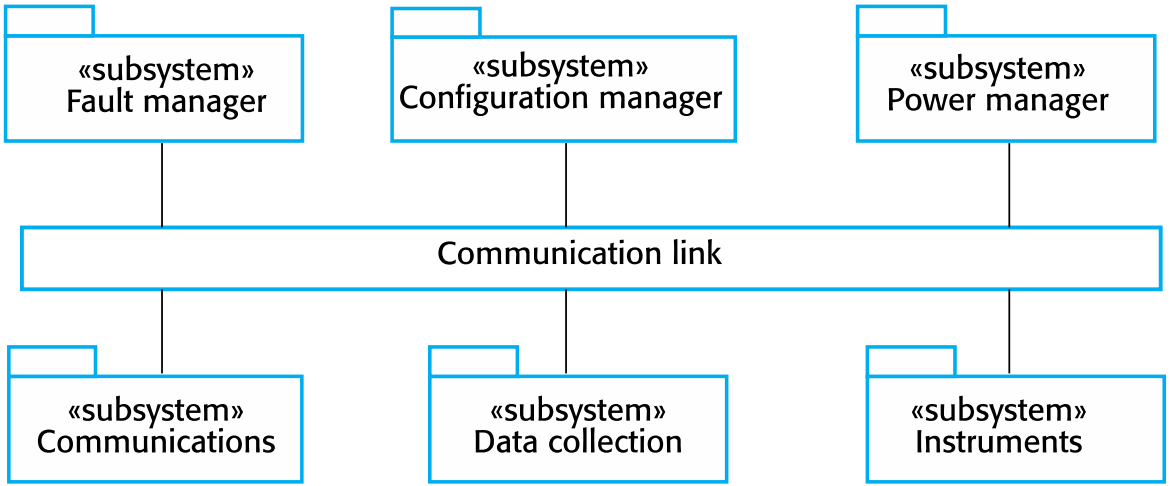
An interaction model is a dynamic model that shows how the system interacts with its environment as it is used.



Object-oriented design using the UML

2. Architectural design

The weather station is composed of independent subsystems that communicate by broadcasting messages on a common infrastructure.

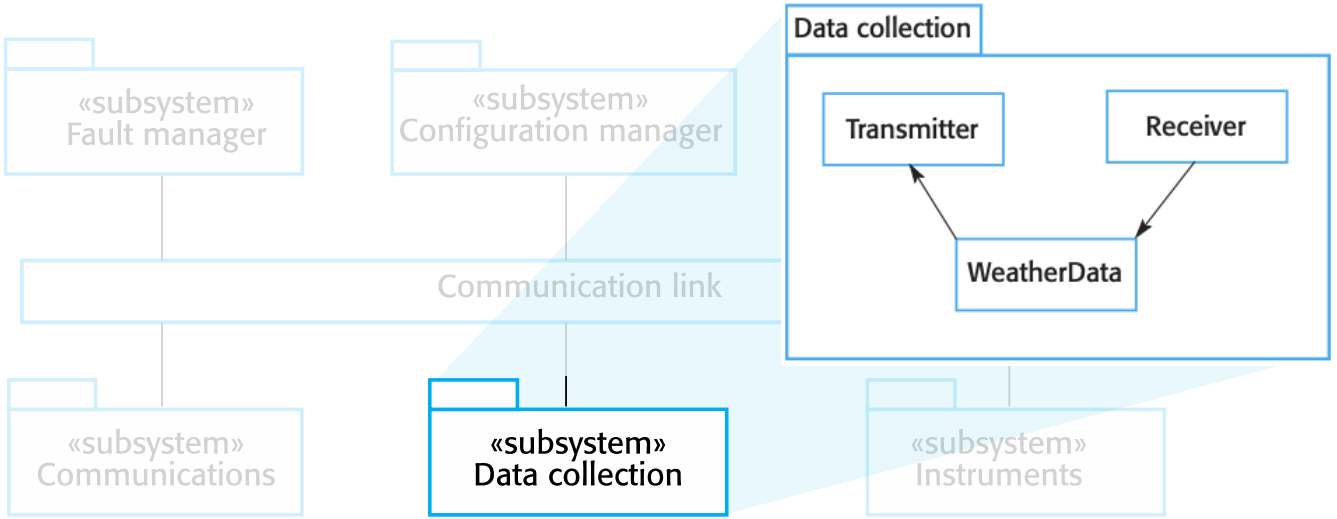


High-level architectural design

Object-oriented design using the UML

2. Architectural design

The weather station is composed of independent subsystems that communicate by broadcasting messages on a common infrastructure.



Data collection subsystem architectural design

Object-oriented design using the UML

3. Object class identification

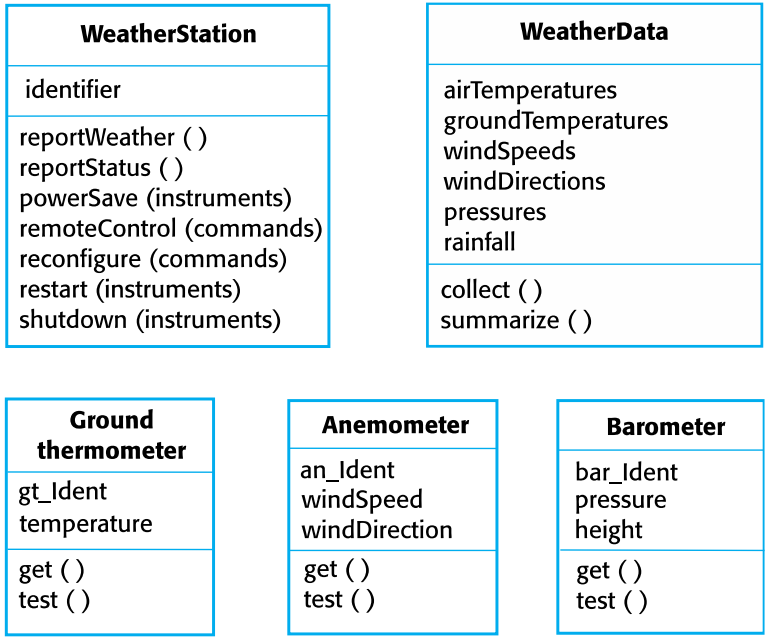
Identifying object classes is often a difficult part of object oriented design.

There is no 'magic formula' for object identification. It relies on the skill, experience and domain knowledge of system designers.

- Object-Oriented Programming (OOP) knowledge
  Difficult to identify classes from abstract concepts.
- Design patterns
  We'll dive into this topic next week.

Object-oriented design using the UML

3. Object class identification



Identified weather station objects

Object-oriented design using the UML

4. Design models

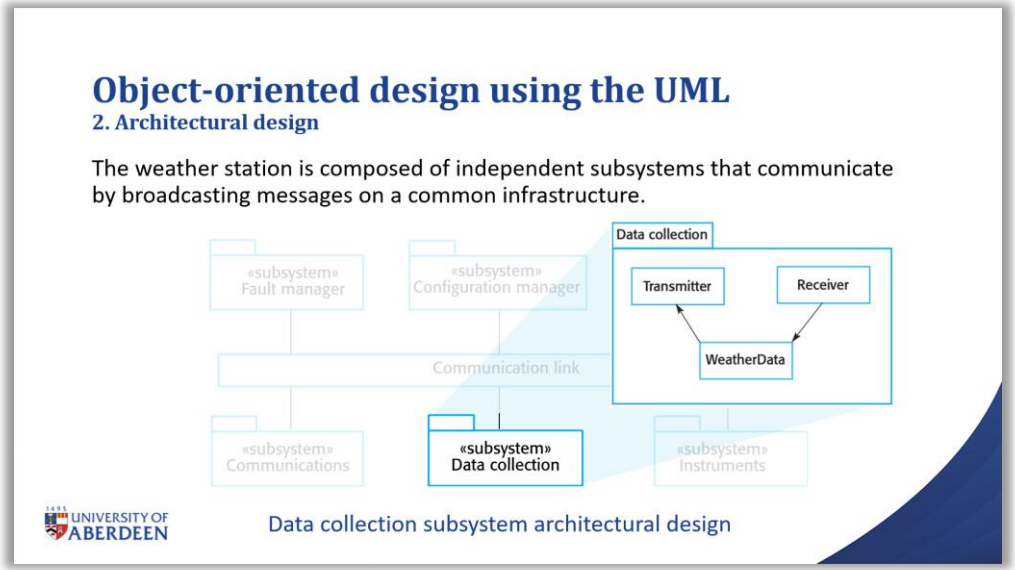
Design models show the objects and object classes and relationships between these entities.

There are two kinds of design model:

- Structural models describe the static structure of the system in terms of object classes and relationships (e.g. UML Class diagram).

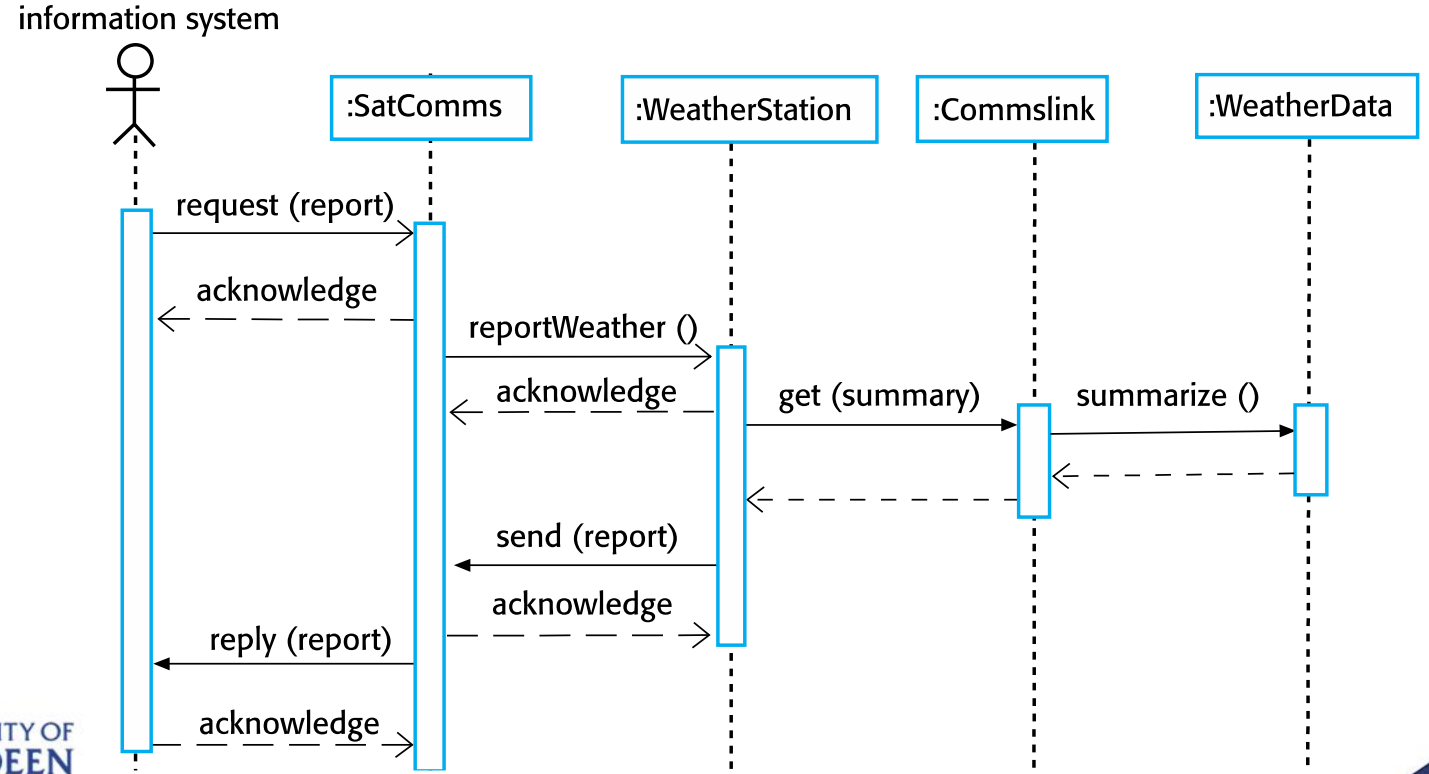
- Dynamic models describe the dynamic interactions between objects (e.g. UML Sequence diagram / UML State diagram)..

Object-oriented design using the UML
4. Design models -- Subsystem models



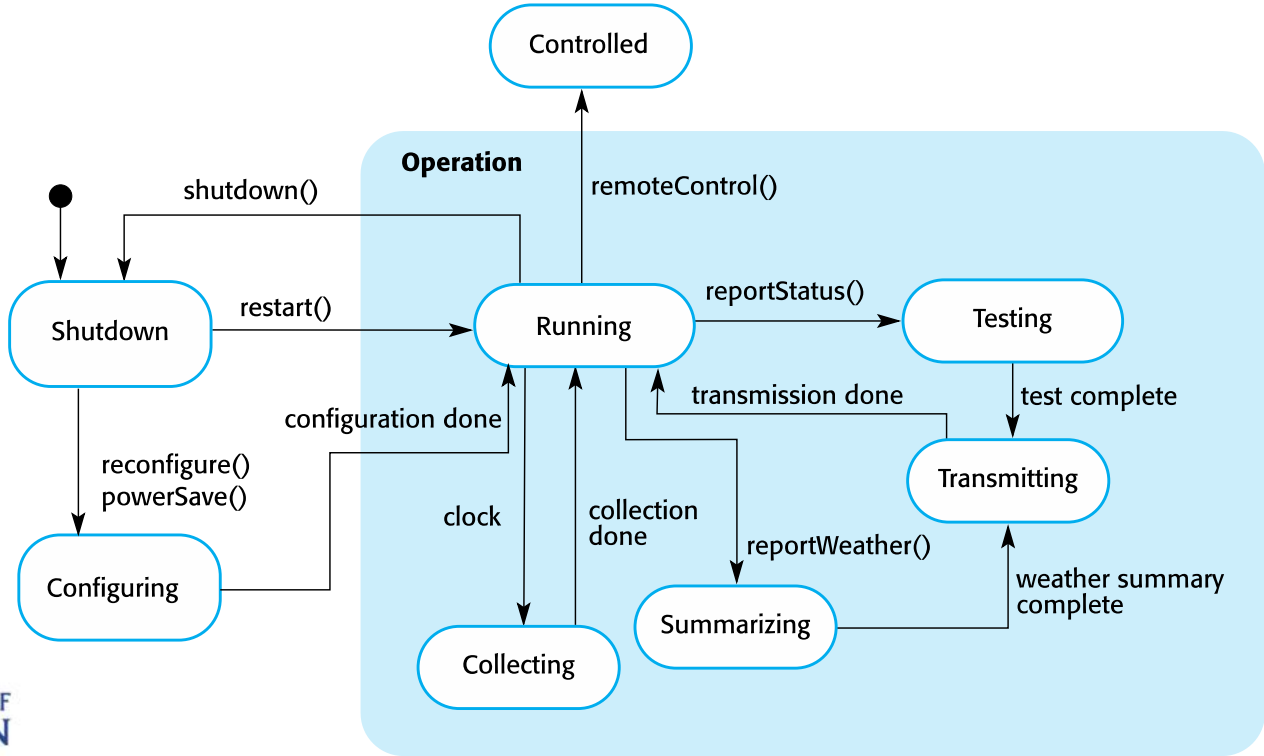
Object-oriented design using the UML

4. Design models -- Sequence models



Object-oriented design using the UML

4. Design models -- State diagrams



Object-oriented design using the UML

5. Interface specification

- Object interfaces have to be specified so that the objects and other components can be designed in parallel.

- Designers should avoid designing the interface representation but should hide this in the object itself.

- Objects may have several interfaces which are viewpoints on the methods provided.

Object-oriented design using the UML

5. Interface specification

- Object interfaces have to be specified so that the objects and other components can be designed in parallel.

- Designers should avoid designing the interface representation but should hide this in the object itself.

- Objects may have several interfaces which are viewpoints on the methods provided.

Object-oriented design using the UML

5. Interface specification

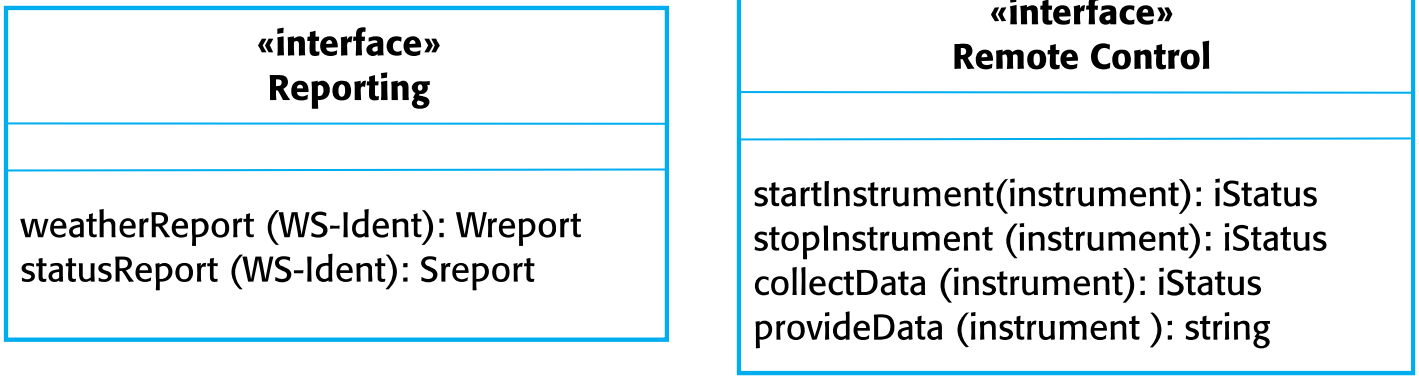
- Object interfaces have to be specified so that the objects and other components can be designed in parallel.

- Designers should avoid designing the interface representation but should hide this in the object itself.

- Objects may have several interfaces which are viewpoints on the methods provided.

Object-oriented design using the UML

5. Interface specification



Object-oriented design using the UML
5. Interface specification

- Interfaces are like contracts that allow independent development.
- Hide internal details; expose only what others need to know.
- Multiple interfaces = multiple perspectives or roles the same object can play.

Case study

A company XYZ keeps track of employees, departments and projects. The company is organized into departments. Each department has a unique name, a unique number, and a particular employee who manages the department. A department may have several locations, and controls a number of projects, each of which has a unique name, a unique number, and a single location. The company stores each employee's name, Social Security number, address, salary, gender and birth date. An employee is assigned to one department, but may work on several projects. The company maintains records of an employee's dependents for insurance purposes, including each dependent's first name, gender, birth date and relationship to the employee.

- Identify four use cases.
- Draw a UML class diagram for one of the four use cases.
- Draw a UML sequence diagram for one of the four use cases.

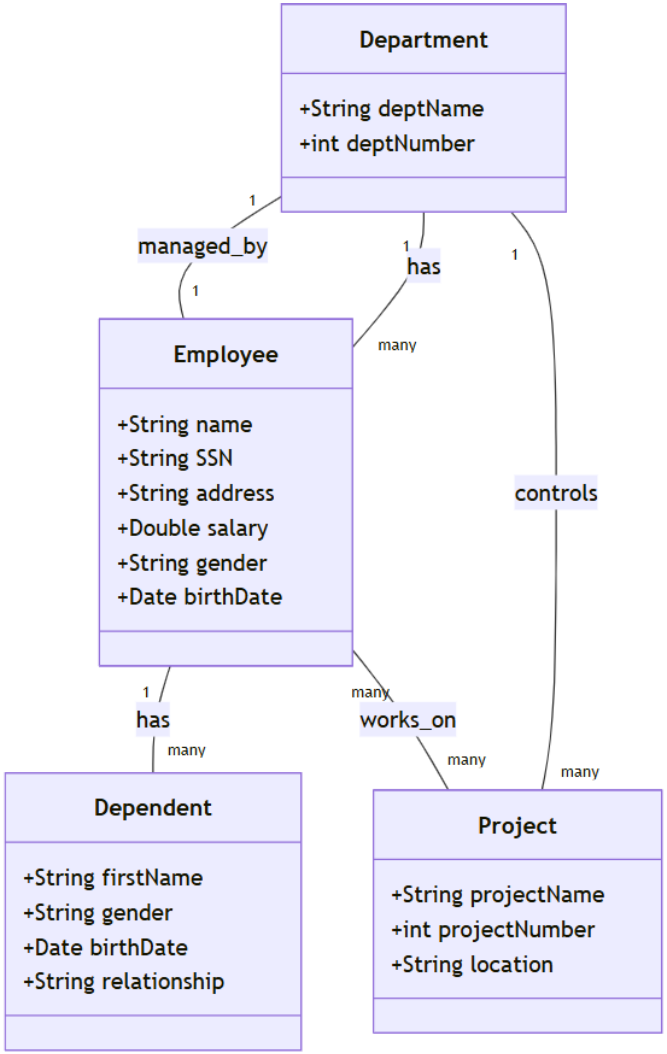
Case study

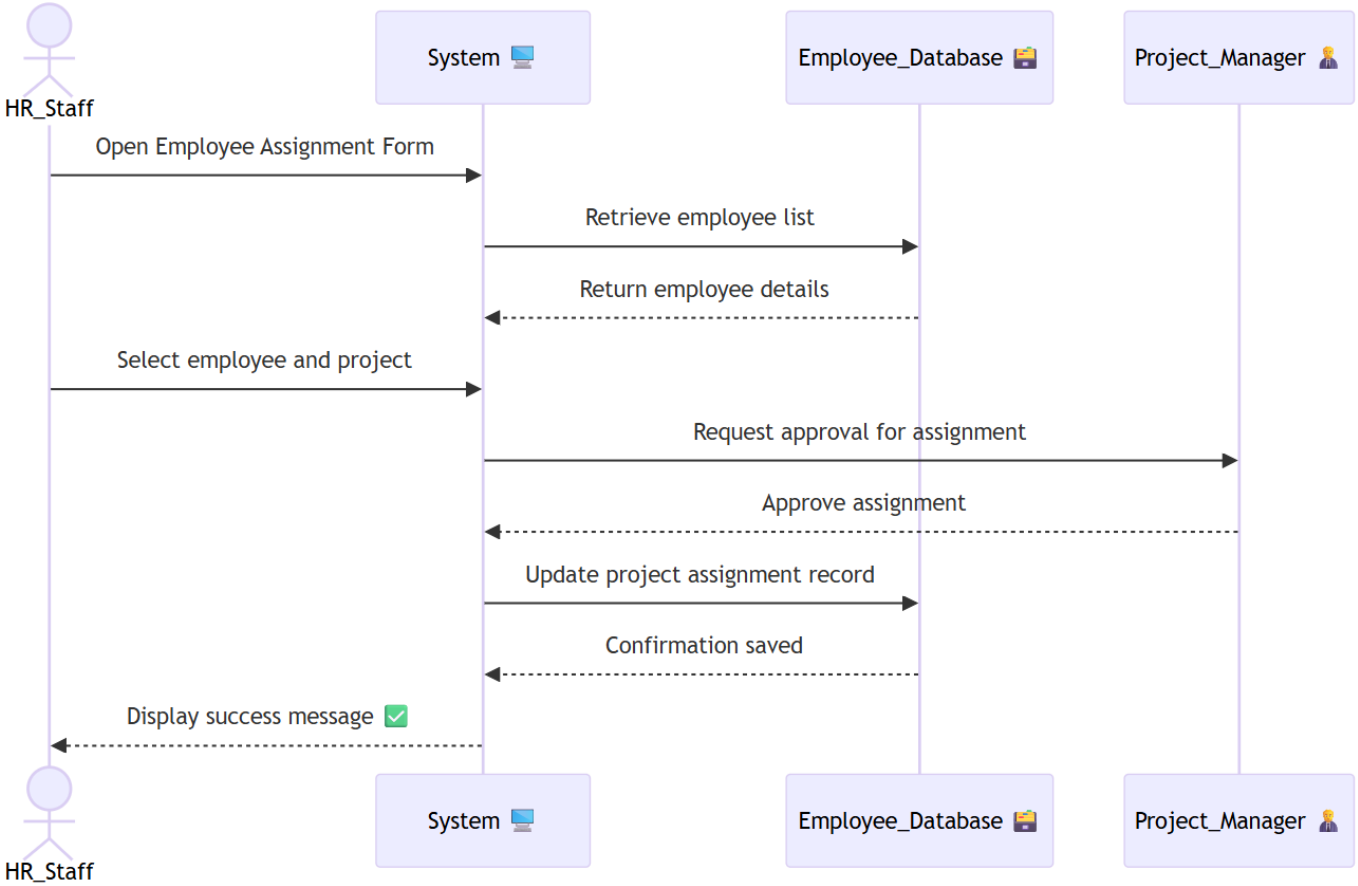
Add a New Employee: HR clerk registers a new employee, assigning them to a department and recording their personal information.

Assign Employee to Project: Project manager links an existing employee to one or more ongoing projects they'll work on.

Update Department Manager: The company appoints a new manager for an existing department.

Add Dependent for Employee: HR staff records a dependent for insurance purposes.


Department
+String deptName
+int deptNumber
1
managed_by
1
1
has
1
many
1
controls
Employee
+String name
+String SSN
+String address
+Double salary
+String gender
+Date birthDate
1
has
many
many
works_on
many
many
Dependent
+String firstName
+String gender
+Date birthDate
+String relationship
Project
+String projectName
+int projectNumber
+String location


HR_Staff
System
Employee_Database
Project_Manager
Open Employee Assignment Form
Retrieve employee list
Return employee details
Select employee and project
Request approval for assignment
Approve assignment
Update project assignment record
Confirmation saved
Display success message
HR_Staff
System
Employee_Database
Project_Manager

Implementation issues

Focus here is not on programming, although this is obviously important, but on other implementation issues that are often not covered in programming texts:

- **Reuse** -- Most modern software is constructed by reusing existing components or systems. When you are developing software, you should make as much use as possible of existing code.
- **Configuration management** -- During the development process, you have to keep track of the many different versions of each software component in a configuration management system.
- **Host-target development** -- Production software does not usually execute on the same computer as the software development environment. Rather, you develop it on one computer (the host system) and execute it on a separate computer (the target system).

Implementation issues
-- Reuse

From the 1960s to the 1990s, most new software was developed from scratch, by writing all code in a high-level programming language.

- The only significant reuse or software was the reuse of functions and objects in programming language libraries.

Implementation issues
-- Reuse levels

The abstraction level

- At this level, you don’t reuse software directly but use knowledge of successful abstractions in the design of your software.

The object level

- At this level, you directly reuse objects from a library rather than writing the code yourself.

The component level

- Components are collections of objects and object classes that you reuse in application systems.

The system level

- At this level, you reuse entire application systems.

Implementation issues
-- Reuse costs

- The costs of the time spent in **looking for** software to reuse and assessing whether or not it meets your needs.
- Where applicable, the costs of **buying** the reusable software. For large off-the-shelf systems, these costs can be very high.
- The costs of **adapting and configuring** the reusable software components or systems to reflect the requirements of the system that you are developing.
- The costs of **integrating** reusable software elements with each other (if you are using software from different sources) and with the new code that you have developed.

Implementation issues
-- Configuration management

- Configuration management is the name given to the general process of managing a changing software system.

- The aim of configuration management is **to support the system integration** process so that all developers can access the project code and documents in a controlled way, find out what changes have been made, and compile and link components to create a system.

Implementation issues
-- Configuration management

Configuration management activities (you need configuration management tool to handle these activities).

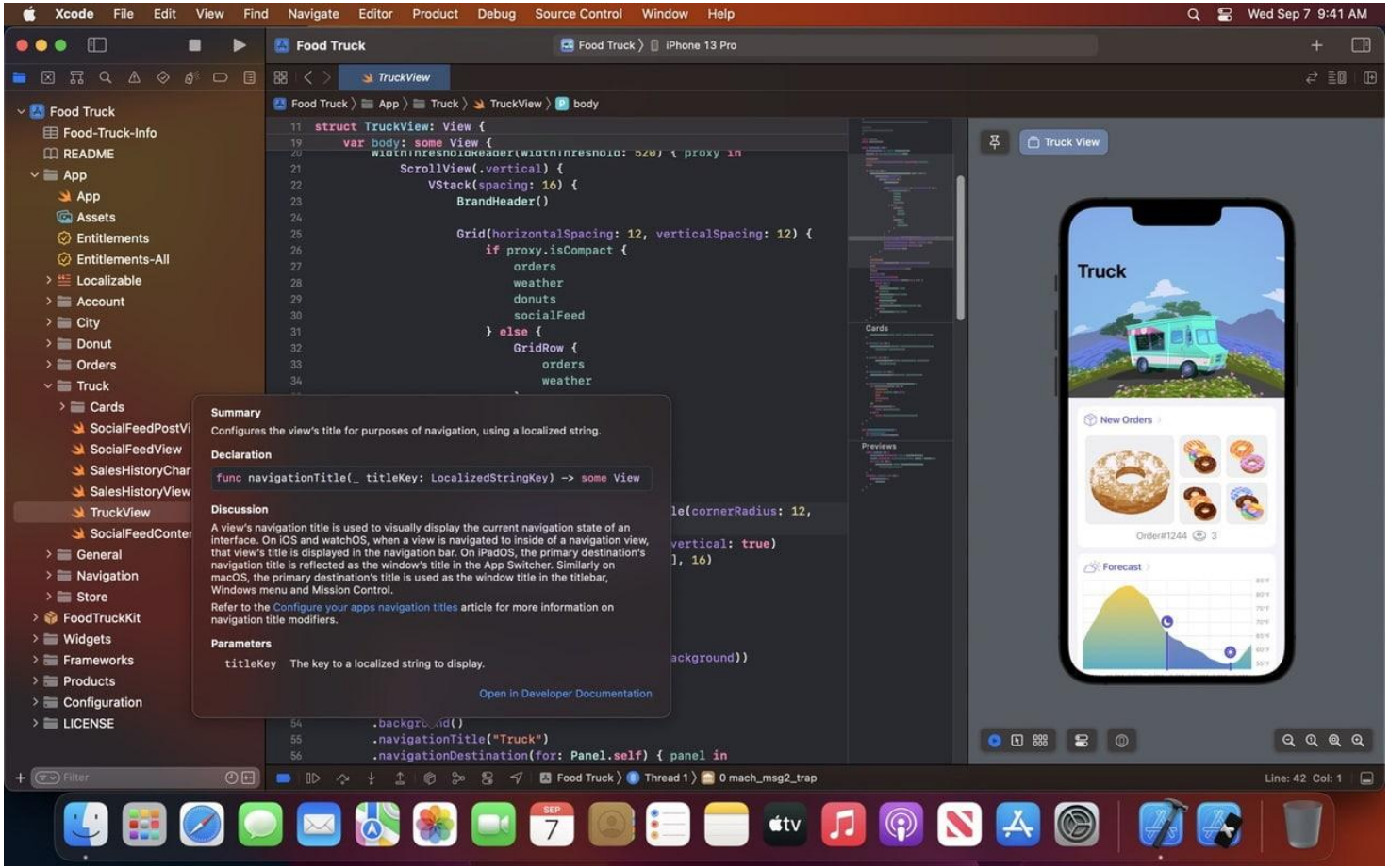
- **Version management**, where support is provided to keep track of the different versions of software components. Version management systems include facilities to coordinate development by several programmers.
- **System integration**, where support is provided to help developers define what versions of components are used to create each version of a system. This description is then used to build a system automatically by compiling and linking the required components.
- **Problem tracking**, where support is provided to allow users to report bugs and other problems, and to allow all developers to see who is working on these problems and when they are fixed.

Implementation issues
-- Host-target development

Most software is developed on one computer (development platform), but runs on a separate machine (execution platform).

Development platform usually has different installed software than execution platform; these platforms may have different architectures.

- If your development platform is same with the execution platform, you can debug your software just on your development platform (that is easy).
- If your development platform is different from the execution platform, you have to use specific develop/debug tools (e.g. simulators or specific hardware).



Xcode (iphone development env) with iphone simulator

Lessons learned

A software architecture is a description of how a software system is organized.

Architectural design decisions include decisions on the type of application, the distribution of the system, the architectural styles to be used.

Architectural patterns are a means of reusing knowledge about generic system architectures. They describe the architecture, explain when it may be used and describe its advantages and disadvantages.

We reviewed the Object-oriented design process using UML.

We introduced the implementation issues. Especially, when developing software, you should always consider the possibility of reusing existing software, either as components, services or complete systems.